Assignment 3

Name: Riddhi Agarwal
Roll Number: 2024CSB046

Problem 1

Question 1

In an online examination platform, the examination scores of 100,000 students are stored sequentially in an array on the server. Once the examination process is completed, the examination authority needs to sort the students' scores efficiently in order to prepare rank lists and perform further analysis. Write a C program to implement the Randomized Quicksort algorithm for sorting the array of students' scores. The program should consider and handle the following three input scenarios:

i) Best-case-like scenario: when the array of scores is already sorted in ascending order.

ii) Worst-case-like scenario for deterministic approaches: when the array is sorted in reverse (descending) order.

iii) Average-case scenario: when the array elements are in a random, unsorted order.

Finally, count the total number of comparisons performed during the sorting process and measure the execution time required to complete the sorting operation.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

long long comparisons = 0;

void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++)
    {
```

```
        comparisons++;
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

int randomizedPartition(int arr[], int low, int high)
{
    int random = low + rand() % (high - low + 1);
    swap(&arr[random], &arr[high]);
    return partition(arr, low, high);
}

void randomizedQuickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = randomizedPartition(arr, low, high);
        randomizedQuickSort(arr, low, pi - 1);
        randomizedQuickSort(arr, pi + 1, high);
    }
}

void generateData(int arr[], int n, int type)
{
    for (int i = 0; i < n; i++)
    {
        if (type == 1)
            arr[i] = i;
        else if (type == 2)
            arr[i] = n - i;
        else
            arr[i] = rand() % n;
    }
}

int main()
{
    int n = 100000;
    int *arr = (int *)malloc(n * sizeof(int));
    if (arr == NULL)
    {
        printf("Memory allocation failed");
        return 1;
    }

```

```c
    srand(time(0));
    clock_t start, end;
    double cpu_time_used;

    printf("%-20s %-20s %-20s\n", "Scenario", "Comparisons", "Time");

    char *scenarios[] = {"Best", "Worst", "Average"};
    for (int t = 1; t <= 3; t++)
    {
        generateData(arr, n, t);

        comparisons = 0;
        start = clock();
        randomizedQuickSort(arr, 0, n - 1);
        end = clock();

        cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
        printf("%-20s %-20lld %-20f\n", scenarios[t - 1], comparisons, cpu_time_used);
    }

    free(arr);
    return 0;
}
```

Output

```
Scenario            Comparisons         Time
Best                2022834             0.021000
Worst               2038447             0.018000
Average             1999523             0.021000
```

Figure 1: Output for Problem 1.

Analysis

- **Randomized Pivot Selection:** Unlike standard Quicksort, which exhibits $O(n^2)$ complexity on sorted or reverse-sorted inputs, the Randomized Quicksort maintains $O(n \log n)$ performance. By picking a random pivot, the algorithm avoids consistently poor partitions, effectively turning any input distribution into an average-case scenario. Thus Randomized Quicksort is highly effective for this online examination platform because it provides consistent, predictable performance regardless of how the students' scores are initially ordered, preventing the server from experiencing significant slowdowns during peak analysis periods.

Problem 2

Question 2

Write a C program to study the impact of duplicate keys on the performance of standard quicksort and randomized quicksort. The program should generate three integer arrays, each of size 100,000, with varying proportions of duplicate elements in order to simulate different real-world data distributions. The arrays should be constructed as follows:

i) Array-I: Contains approximately 90% duplicate values, representing highly repetitive data.

ii) Array-II: Contains approximately 50% duplicate values, representing moderately repetitive data.

iii) Array-III: Contains approximately 10% duplicate values, representing data with low repetition.

After generating the three arrays, implement and apply the Standard Quicksort (a fixed pivot selection strategy) and Randomized Quicksort (a random pivot selection strategy). For each combination of array type and sorting algorithm, the program must count the total number of key comparisons performed during sorting, measure the maximum recursion depth encountered during execution, and measure the execution time required to complete the sorting process. Finally, analyze and compare the performance of Standard Quicksort and Randomized Quicksort across all three data distributions. Plot appropriate 2D graphs (such as input distribution vs. execution time, number of comparisons, and recursion depth) to visually illustrate the effect of duplicate values on the efficiency and stability of both algorithms.

Code

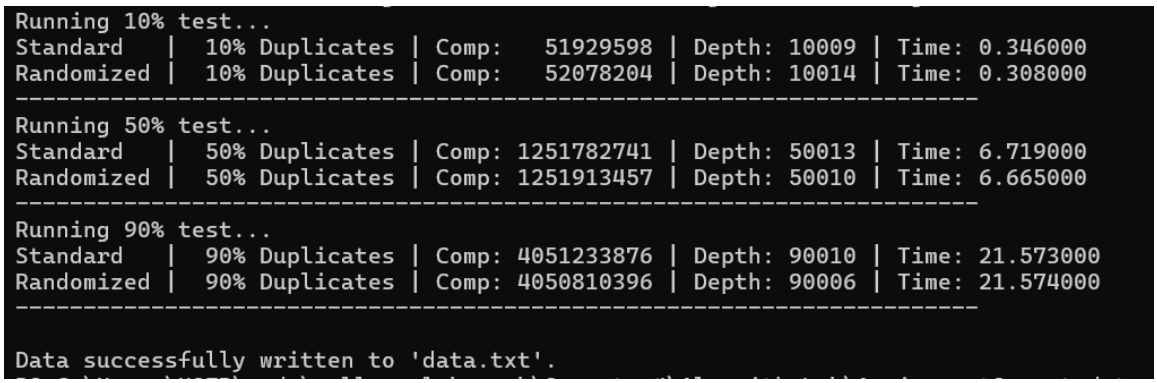
```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

long long comparisons = 0;
int max_recursion_depth = 0;

void swap(int* a, int* b) {
    int t = *a;
    *a = *b;
    *b = t;
}

void generateDuplicateArray(int arr[], int n, int percent_duplicates)
    {
    for(int i = 0; i < n; i++) arr[i] = rand() % n;

    int limit = (n * percent_duplicates) / 100;
    int duplicate_val = rand() % n;
    for(int i = 0; i < limit; i++) {
        arr[i] = duplicate_val;
```

```
    }

    for (int i = 0; i < n; i++) {
        int r = rand() % n;
        swap(&arr[i], &arr[r]);
    }
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
        comparisons++;
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

int randomizedPartition(int arr[], int low, int high) {
    int random = low + rand() % (high - low + 1);
    swap(&arr[random], &arr[high]);
    return partition(arr, low, high);
}

void quickSort(int arr[], int low, int high, int isRandom, int depth)
    {
    if (depth > max_recursion_depth) max_recursion_depth = depth;

    if (low < high) {
        int pi;
        if (isRandom) pi = randomizedPartition(arr, low, high);
        else pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1, isRandom, depth + 1);
        quickSort(arr, pi + 1, high, isRandom, depth + 1);
    }
}

void runTest(int arr[], int n, int duplicate_pct, FILE *dataFile) {
    int *copy = (int*)malloc(n * sizeof(int));
    clock_t start, end;

    double std_time, rand_time;
    long long std_comp, rand_comp;
    int std_depth, rand_depth;

    for(int i=0; i<n; i++) copy[i] = arr[i];
```

```
    comparisons = 0;
    max_recursion_depth = 0;

    start = clock();
    quickSort(copy, 0, n - 1, 0, 0);
    end = clock();

    std_time = ((double)(end-start))/CLOCKS_PER_SEC;
    std_comp = comparisons;
    std_depth = max_recursion_depth;

    for(int i=0; i<n; i++) copy[i] = arr[i];
    comparisons = 0;
    max_recursion_depth = 0;

    start = clock();
    quickSort(copy, 0, n - 1, 1, 0);
    end = clock();

    rand_time = ((double)(end-start))/CLOCKS_PER_SEC;
    rand_comp = comparisons;
    rand_depth = max_recursion_depth;

    printf("Standard   | %3d%% Duplicates | Comp: %10lld | Depth: %5d | Time: %f\n",
            duplicate_pct, std_comp, std_depth, std_time);
    printf("Randomized | %3d%% Duplicates | Comp: %10lld | Depth: %5d | Time: %f\n",
            duplicate_pct, rand_comp, rand_depth, rand_time);
    printf("-----------------------------------------------------------------------\n");

    fprintf(dataFile, "%d %f %f %lld %lld %d %d\n",
            duplicate_pct, std_time, rand_time, std_comp, rand_comp, std_depth, rand_depth);

    free(copy);
}

int main() {
    int n = 100000;
    int *arr = (int*)malloc(n * sizeof(int));
    srand(time(0));

    FILE *fp = fopen("data.txt", "w");
    if (fp == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

```

```c
    fprintf(fp, "# Duplicates Std_Time Rand_Time Std_Comp Rand_Comp Std_Depth Rand_Depth\n");

    printf("Running 10%% test...\n");
    generateDuplicateArray(arr, n, 10);
    runTest(arr, n, 10, fp);

    printf("Running 50%% test...\n");
    generateDuplicateArray(arr, n, 50);
    runTest(arr, n, 50, fp);

    printf("Running 90%% test...\n");
    generateDuplicateArray(arr, n, 90);
    runTest(arr, n, 90, fp);

    fclose(fp);
    free(arr);

    printf("\nData successfully written to 'data.txt'.\n");
    return 0;
}
```

Output



```
Running 10% test...
Standard   |  10% Duplicates | Comp:   51929598 | Depth: 10009 | Time: 0.346000
Randomized |  10% Duplicates | Comp:   52078204 | Depth: 10014 | Time: 0.308000
-----------------------------------------------------------------------
Running 50% test...
Standard   |  50% Duplicates | Comp: 1251782741 | Depth: 50013 | Time: 6.719000
Randomized |  50% Duplicates | Comp: 1251913457 | Depth: 50010 | Time: 6.665000
-----------------------------------------------------------------------
Running 90% test...
Standard   |  90% Duplicates | Comp: 4051233876 | Depth: 90010 | Time: 21.573000
Randomized |  90% Duplicates | Comp: 4050810396 | Depth: 90006 | Time: 21.574000
-----------------------------------------------------------------------

Data successfully written to 'data.txt'.
```

Figure 2: Output for Problem 2.

Analysis

Based on the experimental data and the generated plots, we can draw the following conclusions regarding the impact of duplicate keys:

- **Comparison Count and Execution Time:** Both Standard and Randomized Quicksort show a significant increase in comparisons and execution time as the percentage of duplicate elements increases. This is because the simple partition logic used (`arr[j] <= pivot`) fails to group all duplicates together, leading to highly unbalanced partitions (skewed towards one side) even with a randomized pivot.

- **Recursion Depth:** The recursion depth grows linearly with the percentage of duplicates. For 90% duplicates, the depth reaches nearly 90,000, which is $O(n)$. This indicates

that the presence of many duplicates pushes Quicksort toward its worst-case complexity of $O(n^2)$.

- **Standard vs. Randomized:** In this specific scenario of heavy duplicates, randomization provides little to no advantage over standard quicksort because the "bad" partition is caused by the values themselves (all being equal to the pivot) rather than the position of the pivot.

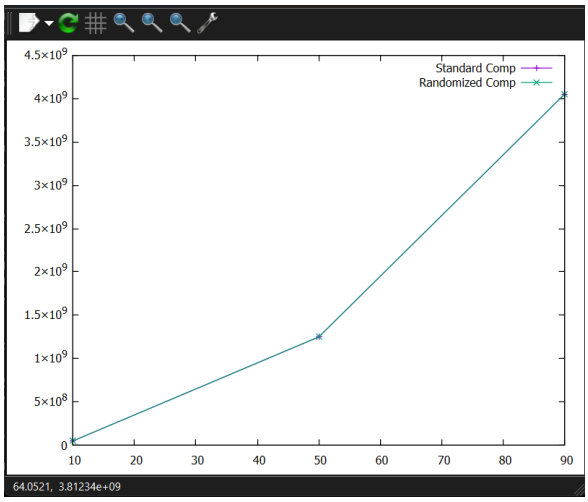


Figure 3: Duplicates vs. Comparisons

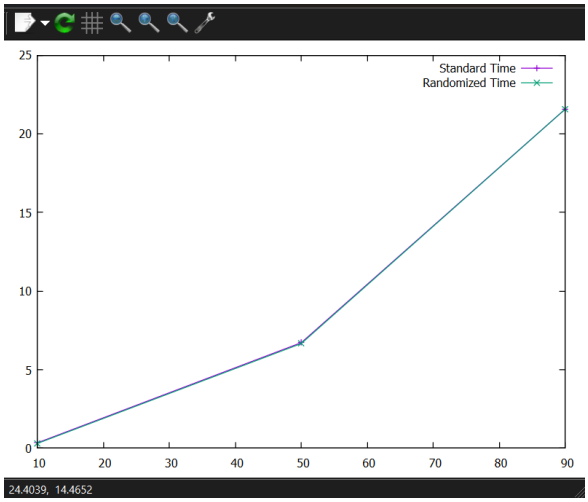


Figure 4: Duplicates vs. Execution Time

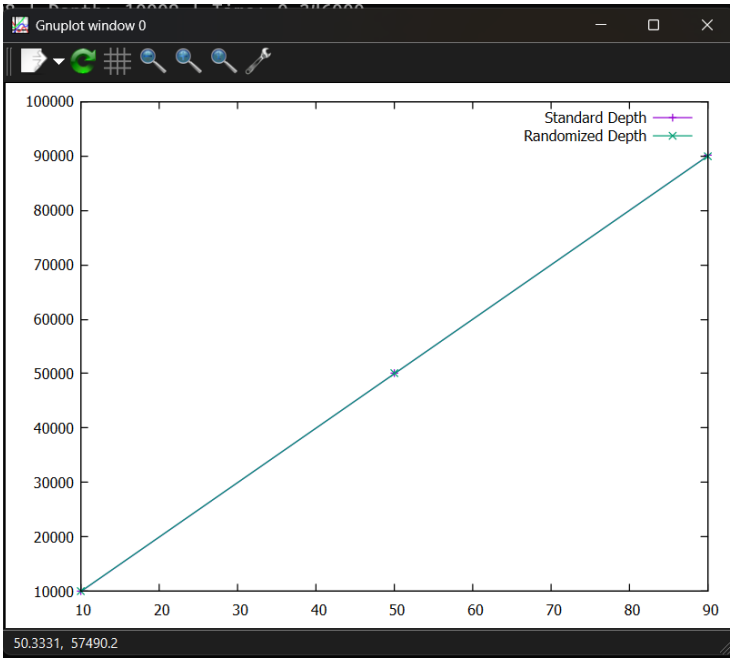


Figure 5: Duplicates vs. Max Recursion Depth

Problem 3

Question 3

Consider an integer array of size 100,000 in which approximately 80% of the elements are already in nearly sorted order, while the remaining elements are randomly positioned. Such data distributions are common in real-world applications. Write a C program to implement both the Quicksort and Merge Sort algorithms for sorting the given array. For both sorting technique, Count the total number of key comparisons performed during execution, measure the maximum recursion depth reached during the sorting process, measure and record the execution time required to complete the sorting operation. Based on a comparative analysis of these performance metrics, justify which algorithm is more suitable for the given input conditions.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

long long comparisons = 0;
int max_depth = 0;

void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

void generateNearlySorted(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        arr[i] = i;

    int start_random = (int)(n * 0.8);
    for (int i = start_random; i < n; i++)
    {
        int r = start_random + rand() % (n - start_random);
        swap(&arr[i], &arr[r]);
    }
    for (int i = 0; i < n / 20; i++)
    {
        int a = rand() % n;
        int b = rand() % n;
        swap(&arr[a], &arr[b]);
    }
}

void merge(int arr[], int l, int m, int r, int depth)
```

```
{
    if (depth > max_depth)
        max_depth = depth;
    int n1 = m - l + 1;
    int n2 = r - m;
    int *L = (int *)malloc(n1 * sizeof(int));
    int *R = (int *)malloc(n2 * sizeof(int));

    for (int i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
    {
        comparisons++;
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }
    while (i < n1)
        arr[k++] = L[i++];
    while (j < n2)
        arr[k++] = R[j++];
    free(L);
    free(R);
}

void mergeSort(int arr[], int l, int r, int depth)
{
    if (l < r)
    {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m, depth + 1);
        mergeSort(arr, m + 1, r, depth + 1);
        merge(arr, l, m, r, depth + 1);
    }
}

int partition(int arr[], int low, int high)
{
    int r = low + rand() % (high - low + 1);
    swap(&arr[r], &arr[high]);
    int pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j < high; j++)
    {
        comparisons++;
        if (arr[j] < pivot)
```

```
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quickSort(int arr[], int low, int high, int depth)
{
    if (depth > max_depth)
        max_depth = depth;
    if (low < high)
    {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1, depth + 1);
        quickSort(arr, pi + 1, high, depth + 1);
    }
}

int main()
{
    int n = 100000;
    int *arr = (int *)malloc(n * sizeof(int));
    int *copy = (int *)malloc(n * sizeof(int));
    srand(time(0));

    generateNearlySorted(arr, n);
    for (int i = 0; i < n; i++)
        copy[i] = arr[i];

    comparisons = 0;
    max_depth = 0;
    clock_t start = clock();
    mergeSort(arr, 0, n - 1, 0);
    clock_t end = clock();
    printf("Merge Sort | Comp: %10lld | Max Depth: %d | Time: %f\n",
           comparisons, max_depth, ((double)(end - start)) /
    CLOCKS_PER_SEC);

    comparisons = 0;
    max_depth = 0;
    start = clock();
    quickSort(copy, 0, n - 1, 0);
    end = clock();
    printf("Quicksort  | Comp: %10lld | Max Depth: %d | Time: %f\n",
           comparisons, max_depth, ((double)(end - start)) /
    CLOCKS_PER_SEC);

```

```
    printf("From the above analysis, we can conclude that for the given input conditions, Randomized Quicksort is more suitable as it performs faster.\n");

    free(arr);
    free(copy);
    return 0;
}
```

Output

```
Merge Sort | Comp:    1154974 | Max Depth: 17 | Time: 0.043000
Quicksort  | Comp:    2111769 | Max Depth: 42 | Time: 0.018000
From the above analysis, we can conclude that for the given input conditions, Randomized Quicksort is more suitable as it performs faster.
```

Figure 6: Output for Problem 3.

Analysis

The performance metrics for the nearly sorted array (80% sorted) reveal distinct characteristics of the two algorithms:

- **Execution Time:** Randomized Quicksort (~0.018s) is significantly faster than Merge Sort (~0.043s). While both have an average-case time complexity of $O(n \log n)$, Quicksort has smaller constant factors and better cache locality, making it more efficient in practice for this distribution.
- **Comparisons:** Merge Sort performs fewer comparisons (~1.15 million) compared to Quicksort (~2.11 million). Merge Sort's comparison count is very stable and follows the $n \log n$ curve strictly, whereas Quicksort's comparisons vary based on pivot selection.
- **Recursion Depth:** Merge Sort maintains a very low and stable recursion depth (17), which is $\lceil \log_2(100,000) \rceil$. Randomized Quicksort has a higher depth (42) but remains well within logarithmic bounds, proving that the randomized pivot effectively prevents the $O(n)$ depth associated with nearly sorted data in deterministic Quicksort.
- **Conclusion:** For this real-world scenario of nearly sorted data, **Randomized Quicksort** is the more suitable choice due to its superior execution speed and lower memory overhead (as it sorts in-place, unlike Merge Sort which requires $O(n)$ auxiliary space).